

Programación Avanzada
Grado en Ingeniería Informática - Curso 2026/2027

EPD 3: JavaScript Básico

IMPORTANTE: Las entregas de este guión deben respetar lo establecido en el documento "Instrucciones de entregas de EPD", suponiendo en caso contrario penalizaciones en las calificaciones obtenidas. Consulte dicho documento antes de realizar una entrega.

Objetivos

- Conocer los elementos básicos del lenguaje JavaScript.
- Crear aplicaciones web interactivas con *scripts* básicos.

Resultados de aprendizaje

1. Escribir programas en JavaScript con la sintaxis correcta.
2. Identificar dónde colocar el código (inline, script interno o archivo externo) y evaluar su impacto.
3. Usar métodos básicos de entrada y salida para interactuar con el usuario.
4. Declarar y manejar variables de tipos primitivos.
5. Convertir valores entre tipos numéricos, cadenas y booleanos.
6. Aplicar operadores aritméticos, lógicos y de comparación para transformar datos.
7. Implementar programación estructurada mediante secuencias, decisiones y bucles.
8. Crear y manipular vectores unidimensionales.
9. Crear y manipular vectores bidimensionales.
10. Ordenar vectores con distintos criterios.
11. Añadir y eliminar elementos de un vector usando métodos del lenguaje.
12. Manipular cadenas con las funciones estándar.
13. Definir funciones para modularizar tareas.
14. Usar parámetros para dar información a funciones.
15. Devolver resultados desde funciones y reutilizarlos.
16. Trabajar con números usando la librería matemática de JavaScript.

Conceptos

1. Elementos básicos de JavaScript:

JavaScript es un lenguaje de programación interpretado y orientado a prototipos (no confundir con la orientación a objetos) que permite, entre otras cosas, crear interfaces web interactivas de forma sencilla. Una de las ventajas más importantes que ofrece es que su código se ejecuta directamente en el propio navegador del cliente (esto es, empleando los recursos computacionales del usuario) con la consecuente descarga de trabajo del servidor web.

Los programas escritos en JavaScript se incrustan dentro de las páginas HTML. Se envían con éstas a la máquina del cliente cuando éste solicita cada página, o posteriormente como archivos asociados a la página, de forma similar a como ocurre con las imágenes y las hojas de estilo. El navegador del usuario será el encargado de interpretar y ejecutar los *scripts* incluidos en la página cuando sea necesario.

2. Incrustación de un script en un documento HTML:

El código de un programa escrito en JavaScript se puede incluir dentro de una página HTML de dos formas:

1. Entre las etiquetas <script type="text/javascript"> y </script>
2. Incluyendo un archivo que contiene un *script*, de la forma <script type="text/javascript" src = "nombreScript.js"></script> , donde "nombreScript.js" es el nombre del archivo que contiene el *script*. Debido a la definición del estándar HTML por parte de W3C, se deberá usar la forma no vacía aunque el bloque dentro de la etiqueta *script* no tenga contenido.

En el caso de que empleemos la segunda opción para incluir código JavaScript, tendremos que crear un archivo, aparte de la página web, que contenga el código a incluir.

Podemos crear un archivo para un *script* en un proyecto web de Netbeans de una forma sencilla. Pulse el botón derecho del ratón sobre el nodo del proyecto en el que desea crear el nuevo *script* y seleccione la opción "JavaScript File".

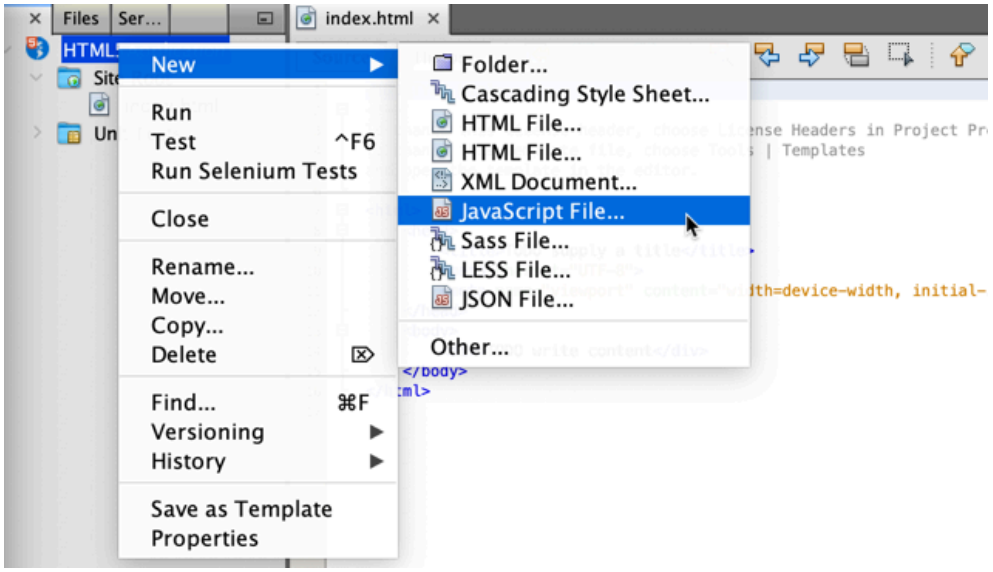


Figura 1 - Selección del archivo JavaScript

En la ventana que se le muestra a continuación deberá dar un nombre al *script*. Escriba el nombre y pulse en finalizar.

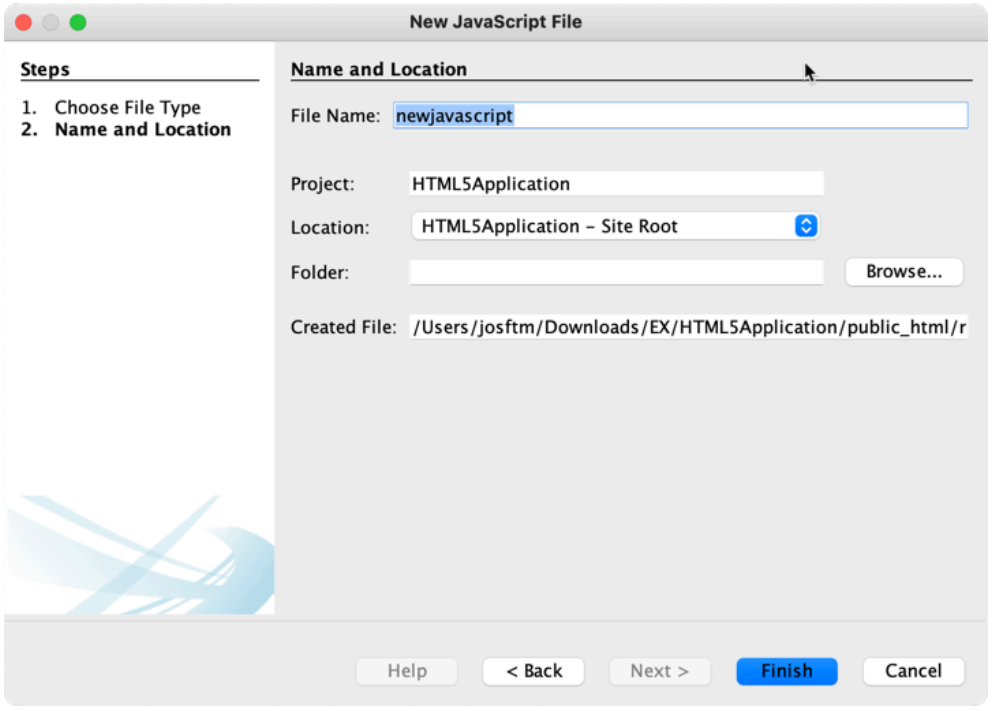


Figura 2 - Asignación de nombre al script

Seguidos los pasos anteriores, aparecerá una nueva pestaña que permitirá editar el archivo que contiene el nuevo *script* creado. Este archivo se incorporará al nodo correspondiente en el proyecto para el cual se ha realizado la creación del *script*.

Para poder depurar *scripts* podemos utilizar las herramientas de desarrollador web de Firefox o Chrome, concretamente la consola web y el depurador.

2.1 Acceso al panel de depuración en Firefox:

Estas opciones se abrirán seleccionándolas desde la opción del menú *Desarrollo - Desarrollador web*, y nos aparecerán unas ventanas similares a las siguientes:

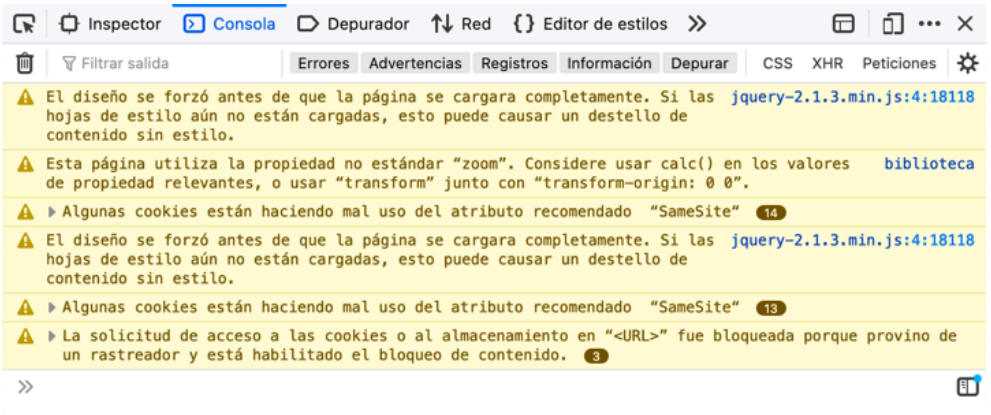


Figura 3 - Consola web de Firefox

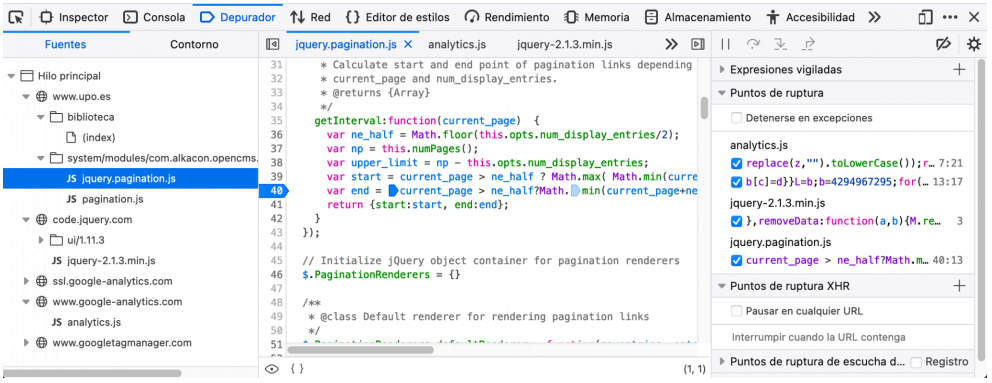


Figura 4 - Depurador de Firefox

Puede encontrar más información sobre la depuración de código JavaScript en el navegador Firefox en [2].

2.2 Acceso al panel de depuración en Chrome:

En Google Chrome, las herramientas de desarrollo se abren desde el menú Más herramientas - Herramientas para desarrolladores. Al hacerlo, aparecerá una interfaz con varias pestañas (Elements, Console, Sources, Network, etc.) que permiten inspeccionar el código HTML, depurar scripts JavaScript y analizar el rendimiento de la página en tiempo real.

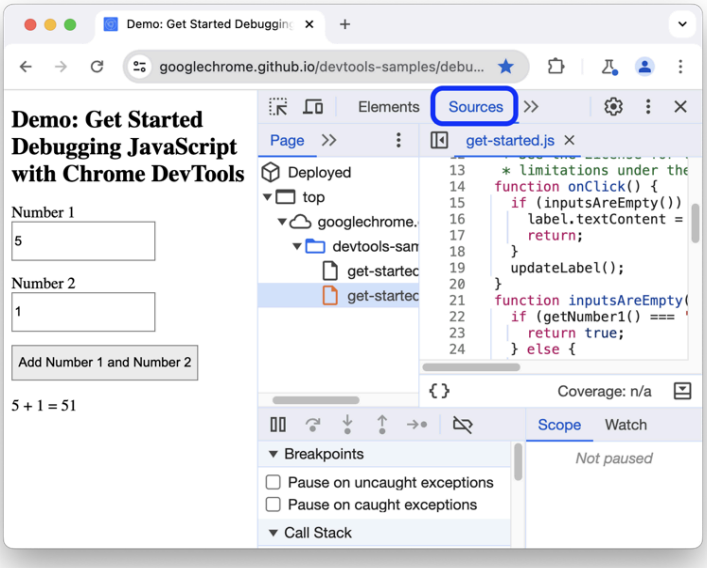


Figura 5 - Depurador de Chrome

Bibliografía Básica

1. JavaScript for Absolute Beginners
McNavage, Terry, Berkley, CA: Apress: Imprint: Apress: 2010
https://eureka.upo.es/permalink/34CBUA_UPO/1qnd2ds/alma991001864587004989

2. Debugging JavaScript
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/debugger

3. Herramientas para desarrolladores de Chrome
https://developer.chrome.com/docs/devtools/open?hl=es-419

Experimentos

E1. (15 mins.) - Resultados de aprendizaje: 1, 2, 3, 4, 7, 13, 14 y 15

Observe el comportamiento del navegador Firefox ante códigos JavaScript que contienen errores de diferentes tipos:

a) Errores en tiempo de carga: Se producen cuando el navegador puede determinar que el *script* tiene un error al cargar el código. Normalmente proceden de errores de sintaxis, como los que contiene este código:

```
<html>
<head>
        <script type="text/javascript">
        <!--
                function hola(){
                        const cadena = ¡Hola mundo!
                        document.write(cadena);
                }
        //-->
        </script>
</head>
```

```
<body>
	<script type="text/javascript">
	<!--
		hola();
	//-->
	</script>
</body>
</html>
```

b) Errores en tiempo de ejecución: Este tipo de errores se produce cuando el navegador detecta un error al ejecutar una sentencia del código:

```
<html>
<head>
	<script type="text/javascript">
	<!--
		function hola(nombre){
			document.write("¡Hola " + Nombre + "!");
		}
	//-->
	</script>
</head>
<body>
	<script type="text/javascript">
	<!--
		const nombre = prompt("Dime tu nombre","");
		hola(nombre);
	//-->
	</script>
</body>
</html>
```

c) Errores lógicos: Aunque el programa no contenga ningún error de los anteriores y, por tanto, esté perfectamente escrito en función de las reglas del lenguaje de programación, puede contener errores lógicos. Este tipo de errores supone que el programa no se comporte como el programador esperaba. En estos casos, la función *console.log( )* nos permitirá ver qué está ocurriendo en el código. Observe qué ocurre en el siguiente código:

```
<html>
<head>
	<script type="text/javascript">
	<!--
		function power(numero,p){
			//console.log("Calculando potencia de "+ p + " de " + numero);
			return numero * power(numero,p-1);
			}
	//-->
	</script>
</head>
<body>
	<script type="text/javascript">
	<!--
		const numero = prompt("¿A que numero le calculamos la potencia?","");
		const potencia = prompt("la potencia ");
		const resultado = power(numero,potencia);
		alert("La potencia de " + potencia + " de " + numero +
			" es " + resultado);
	//-->
	</script>
</body>
</html>
```

E2. (15 mins.) - Resultados de aprendizaje: 1, 2, 3, 13, 14 y 15

Empleando el código anterior, pruebe las funcionalidades del depurador de Firefox y Chrome.

Ejercicios

CONSIDERACIONES:

- **El guión trata sobre la creación de funcionalidades a través de JS.**
- **En esta práctica únicamente se valorarán las competencias relacionadas con JS.**
- **Para la evaluación de esta práctica se escogerá un ejercicio al azar para ser evaluado. La nota total de la práctica dependerá de dicha calificación. Entregue cada ejercicio en una carpeta diferente antes de subir el archivo zip como entrega.**
- **En los siguientes problemas, además de la calidad técnica se valorará la limpieza de código y la optimización del mismo.**
- **Por cada ejercicio, incluya únicamente los archivos HTML y JS necesarios para resolver el ejercicio. Si usa un IDE, no envíe todo el proyecto por cada ejercicio ya que será sancionado.**

Ej1. (45 mins.) Resultados de aprendizaje: 1, 3, 5, 6, 7, 8, 10 y 16

Escriba un programa en JavaScript que permita a un docente registrar de forma dinámica las calificaciones finales obtenidas por los estudiantes de un grupo y ofrezca un informe estadístico completo sobre el rendimiento académico de la clase.

- Entrada de datos:
  - Solicite mediante `prompt()` el número total de calificaciones a registrar (`N`).
  - A continuación, solicite secuencialmente cada una de las `N` notas.
- Validaciones:
  - `N` debe ser un número entero mayor que 0.
  - Cada calificación recogida debe ser un valor numérico en la escala de `0` a `10`, ambos incluídos.
  - Las notas pueden incluir decimales (ej. `7.5`, `9.2`).
  - Si el usuario introduce un dato no numérico o fuera de rango, debe volver a solicitarse hasta que sea válido.
- Clasificación por rendimiento:
  - Sobresaliente: De `8.0` a `10.0` puntos (`8 <= x <= 10`).
  - Notable: De `6.0` a menos de `8.0` puntos (`6 <= x < 8`).
  - Aprobado: De `4.0` a menos de `6.0` puntos (`4 <= x < 6`).
  - Suspenso: Menor a `4.0` puntos (`0 <= x < 4`).
- Cálculos y estadísticas a desarrollar:
  - Calificación media de la clase (redondeada a dos decimales).
  - Calificación máxima y mínima utilizando `Math.max()`[1] y `Math.min()`[2].
  - Para el redondeo use `Math.round()`[3].
  - Porcentaje y recuento de estudiantes en cada uno de los cuatro niveles.
  - Calificaciones ordenadas de mayor a menor (descendente) y de menor a mayor (ascendente). Debe utilizar la función *`sort()`*[4] con una función de comparación numérica `(a, b) => a - b`, ya que el

[1] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/max
[2] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/min
[3] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/round
[4] https://www.w3schools.com/jsref/jsref_sort.asp

ordenamiento por defecto de la función `sort()` es alfabético y ordenaría erróneamente valores numéricos. *Otra opción es usar* `reverse()`[5].

Ejemplo:

```
// Entrada del usuario:
"¿Cuántas calificaciones desea introducir?": 5

        // Calificaciones introducidas y salida con la clasificación:
        Calificación 1: 7.5
                -> Nivel: Notable
        Calificación 2: 9.2
                -> Nivel: Sobresaliente
        Calificación 3: 3.8
                -> Nivel: Suspenso
        Calificación 4: 8.1
                -> Nivel: Sobresaliente
        Calificación 5: 5.5
                -> Nivel: Aprobado

        // Resumen final mostrado:
        Calificación máxima: 9.2
        Calificación mínima: 3.8
        Calificación media: 6.82
        Distribución de niveles:
        - Sobresaliente: 2 estudiantes (40%)
        - Notable: 1 estudiante (20%)
        - Aprobado: 1 estudiante (20%)
        - Suspenso: 1 estudiante (20%)
        Calificaciones ordenadas (Mayor a menor): 9.2, 8.1, 7.5, 5.5, 3.8
        Calificaciones ordenadas (Menor a mayor): 3.8, 5.5, 7.5, 8.1, 9.2
```

Ej2. (35 mins.) Resultados de aprendizaje: 1, 3, 5, 6, 7, 12, 13, 14, 15 y 16
Implemente un programa que calcule el coste de envío de un paquete a partir de los datos introducidos por el usuario mediante `prompt()`.

- Entrada de datos:
  - Peso del paquete en kilogramos.
  - Dimensiones del paquete: largo, ancho y alto en centímetros.
  - Tipo de destino: `LOCAL`, `NACIONAL` o `INTERNACIONAL`.
- Validaciones de entrada:
  - El peso real debe ser un valor numérico (acepta un decimal) mayor que 0 y menor o igual a 30 kg.
  - Cada una de las tres dimensiones debe ser un número entero mayor que 0 y menor o igual a 120 cm.
  - El tipo de destino debe coincidir con alguna de las tres opciones permitidas. Validación insensible a mayúsculas/minúsculas.
  - Si cualquiera de los datos es incorrecto o no numérico, el programa debe mostrar exactamente un mensaje de error indicando el error cometido: `ERROR: <<INDIQUE EL TIPO DE ERROR COMETIDO>>`
- Reglas de cálculo:
  - Peso volumétrico: Se calcula con la fórmula:
    - `pesoVolumetrico = (largo * ancho * alto) / 5000`
  - Peso facturable: Es el mayor valor entre el peso real y el peso volumétrico. Debe redondearse al kilogramo superior mediante la función `Math.ceil()`[6].

[5] https://www.w3schools.com/jsref/jsref_reverse.asp
[6] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/ceil

- Tarifas por kilogramo facturable:
  - Destino Local: 2.50 €/kg.
  - Destino Nacional: 4.00 €/kg.
  - Destino Internacional: 8.50 €/kg.
- Suplemento fijo internacional: Los envíos internacionales tienen un suplemento fijo adicional de 12.00 €.
- Recargo por peso elevado: Si el peso facturable supera los 20 kg, se aplicará un recargo del 15% sobre el subtotal del envío.

- Consideraciones para el desarrollo:
  - Defina la función `calcular(pesoReal, largo, ancho, alto, destino)` que realice las validaciones y cálculos.
  - Si los datos son válidos, la función debe devolver un objeto con el desglose del envío:

```
{
  pesoReal: ...,
  pesoVolumetrico: ...,
  pesoFacturable: ...,
  suplemento: ...,
  recargo: ...,
  precioTotal: ...
}
```

  - Utilice el valor devuelto por la función para presentar en pantalla el resumen formateado con dos decimales (`toFixed(2)`[7]).

Ejemplos:

Ejemplo 1: Envío local

```
Peso del paquete: 3
Largo: 30
Ancho: 20
Alto: 10
Destino: local

Peso real: 3 kg
Peso volumétrico: 1.20 kg
Peso facturable: 3 kg
Precio del envío: 7.50€
```

Ejemplo 2: Uso de peso volumétrico y recargo (> 20 kg)

```
Peso del paquete: 4.3
Largo: 60
Ancho: 40
Alto: 50
Destino: NACIONAL

Peso real: 4.3 kg
Peso volumétrico: 24.00 kg
Peso facturable: 24 kg
Recargo por peso elevado: 14.40€
```

[7] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toFixed

```
Precio del envío: 110.40€
```

Ejemplo 3: Envío internacional con suplemento

```
Peso del paquete: 5
Largo: 40
Ancho: 30
Alto: 20
Destino: internacional

Peso real: 5 kg
Peso volumétrico: 4.80 kg
Peso facturable: 5 kg
Suplemento internacional: 12.00€
Precio del envío: 54.50€
```

Ejemplo 4: Datos incorrectos

```
Peso del paquete: -2
ERROR: DATOS INCORRECTOS
```

Ej3. (45 mins) Resultados de aprendizaje: 1, 3, 6, 7 y 12

Cree un programa que procese códigos de actividades deportivas siguiendo reglas específicas de formateo. Pida una cadena de texto al usuario (que represente un código de actividad) y modifíquela siguiendo los siguientes parámetros:

- Posiciones pares: Deberán convertirse a mayúsculas.
- Posiciones múltiplos de 3: Deberán convertirse a minúsculas.
- Posiciones múltiplos de 5: Deberá reemplazar el carácter por cadena vacía.

Imprima en una alerta el resultado obtenido. Haga uso del método *charAt()* para recuperar los caracteres de una cadena.

Ejemplo de entrada y salida:

```
Entrada: "natacion2025"
Salida: "NAtAc*On2025"

Explicación:
- Posición 0 (n): par : N (mayúscula)
- Posición 1 (a): no cumple reglas : a (sin cambio)
- Posición 2 (t): par : T (mayúscula)
- Posición 3 (a): múltiplo de 3 : a (minúscula)
- Posición 4 (c): par : C (mayúscula)
- Posición 5 (i): múltiplo de 5 : * (asterisco)
- Posición 6 (o): par + múltiplo de 3 : o (minúscula tiene prioridad)
- Posición 7 (n): no cumple reglas : n (sin cambio)
- Posición 8 (2): par : 2 (números no se afectan por mayús/minús)
- Posición 9 (0): múltiplo de 3 : 0 (sin cambio)
- Posición 10 (2): par + múltiplo de 5 : * (asterisco tiene prioridad)
- Posición 11 (5): no cumple reglas : 5 (sin cambio)
```

Ej4. (35 mins.) Resultados de aprendizaje: 1, 2 y 13

Consulte la página https://css-tricks.com/snippets/javascript/ y realice tres páginas distintas que incorporen cada una de ellas una funcionalidad implementada en JavaScript usando al menos tres de los snippets que se ofrecen gratuitamente en

esta página web. Indique la página original de la que ha incorporado el código. Tenga en cuenta que se valorará la complejidad e innovación de la solución.

Ej5. (35 mins.) Resultados de aprendizaje: 1, 4, 6, 7, 8, 11, 12, 13, 14 y 15

Cree una pequeña aplicación en JavaScript para gestionar las actividades deportivas disponibles en un gimnasio. Utilice un vector bidimensional para almacenar información sobre las actividades, como el nombre, el instructor asignado y el número de plazas disponibles. Cada elemento del vector debe ser un objeto que contenga información específica de cada actividad.

Ejemplo:

```
var actividades = [
    { nombre: "Spinning", instructor: "Ana López", plazas: 15 },
    { nombre: "Yoga", instructor: "Carmen Ruiz", plazas: 12 },
    { nombre: "Pilates", instructor: "Miguel Moya", plazas: 10 },
    { nombre: "Zumba", instructor: "Laura Martín", plazas: 20 },
    { nombre: "CrossFit", instructor: "David García", plazas: 8 },
    { nombre: "Aqua Aeróbicos", instructor: "Elena Sánchez", plazas: 18 },
    { nombre: "BodyCombat", instructor: "Carlos Vega", plazas: 16 },
    { nombre: "TRX", instructor: "María González", plazas: 6 }
    // Debe incluir al menos 10 actividades en total
];
```

Se pide implementar un programa que solicite al usuario una de las siguientes opciones:

- Mostrar actividades disponibles
- Reservar plaza en actividad
- Liberar plaza de actividad
- Agregar nueva actividad
- Salir

Para cumplir con lo anterior, implemente las siguientes funciones:

- *`mostrarActividades()`:* Recorre el vector y muestra la información de cada actividad en formato "Nombre: Instructor - Plazas disponibles"
- *`reservarPlaza()`:* Solicita el nombre de una actividad y reduce en 1 las plazas disponibles. Si no hay plazas disponibles o la actividad no existe, muestra un mensaje de error.
- *`liberarPlaza()`:* Solicita el nombre de una actividad e incrementa en 1 las plazas disponibles. Si la actividad no existe, muestra un mensaje de error.
- *`agregarActividad()`:* Solicita nombre de la actividad, instructor y número de plazas, y añade la nueva actividad al array.

Luego de realizar las operaciones anteriores, utiliza la función *`mostrarActividades()`* para mostrar la información:

```
Actividades Disponibles:
Spinning: Ana López - 15 plazas
Yoga: Carmen Ruiz - 12 plazas
Pilates: Miguel Torres - 10 plazas
Zumba: Laura Martín - 20 plazas

¿Qué operación desea realizar?
1. Mostrar actividades
2. Reservar plaza
3. Liberar plaza
4. Agregar actividad
5. Salir

Opción: 2
Ingrese el nombre de la actividad: Yoga
```

```
Plaza reservada con éxito en Yoga.

Actividades Actualizadas:
Spinning: Ana López - 15 plazas
Yoga: Carmen Ruiz - 11 plazas
Pilates: Miguel Torres - 10 plazas
Zumba: Laura Martín - 20 plazas
```

Ej6. (35 mins.) Resultados de aprendizaje: 1, 3, 4, 5, 6, 7, 12, 13, 14, 15 y 16

Cree una página web en JavaScript que permita analizar el contenido de un código QR escaneado por los usuarios al llegar al gimnasio, con el objetivo de verificar y registrar una reserva asociada a una actividad.

El contenido del QR contendrá una URL de reserva con parámetros similares a los siguientes:

http://gympa.upo/reserva?actividad=spinning&duracion=45&id_participante=15

La web deberá:

1) Simular la lectura de un QR (por ejemplo, pidiendo directamente la URL en prompt). **No se debe programar la generación de un QR, se debe simular su lectura. En el interior del QR existirá una URL a analizar.**
2) Extraer y visualizar los parámetros de la URL (actividad, duración, identificador del participante).
3) Validar los datos según los siguientes criterios:
    a) Actividad: Debe ser una de las permitidas (spinning, yoga, pilates, zumba, crossfit, aqua, bodycombat o trx).
    b) Duración: Entre 30 y 120 minutos.
    c) Participantes: Número entre 1 y 50 (inclusive).
4) Calcular el coste total de la reserva aplicando el siguiente cálculo:
    a) Coste base: 1.50 € por participante.
    b) Si la duración es mayor a 30 min: +0.50 €
    c) Si el identificador del participante está entre el 1 y el 15 (inclusive) (es una persona que reserva rápidamente): Se aplica un descuento de un 10%.
5) Mostrar en la página los resultados en forma de tabla:

Parámetro	**Valor**	**Validación**
Actividad	spinning	Correcta
Duración	45	Correcta
Identificador del participante	15	Correcta
Coste total	1.35 €	RESERVA REALIZADA

En caso de errores, se deberá indicar claramente qué parámetros no son válidos y mostrar el mensaje final `RESERVA RECHAZADA`.

Ejemplo de reserva rechazada:

```
http://gympa.upo/reserva?actividad=ballet&duracion=121&id_participante=51
```

Parámetro	**Valor**	**Validación**
Actividad	ballet	Incorrecto - Actividad no reconocida
Duración	121	Incorrecto - Duración máxima

		excedida
Identificador del participante	51	Incorrecto
Coste total		RESERVA RECHAZADA

Ampliación de Bibliografía

1. The book of JavaScript a practical guide to interactive Web pages Thau.San Francisco : No Starch Press, 2006. https://eureka.upo.es/permalink/34CBUA_UPO/1qnd2ds/alma991001829702704989

2. Start Here!: Learn JavaScript, Steve Suehring. Microsoft Press; 2012 https://eureka.upo.es/permalink/34CBUA_UPO/312f63/cdi_proquest_ebookcentral_EBC7117316



Programación Avanzada
Grado en Ingeniería Informática - Curso 2026/2027

EPD 3: JavaScript Básico

Datos de la Práctica

Estimación temporal:

- Parte presencial: 120 minutos.
  - Explicación inicial: 5 minutos.
  - Experimentos: 30 minutos.
  - Ejercicios: 85 minutos.
- Parte no presencial: 270 minutos.
  - Lectura y estudio del guión y bibliografía básica: 45 minutos
  - Ejercicios: 225 minutos